GUIDE 09

Connect Frontend and Backend with Redux Toolkit and Debug the Full Stack

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Connect React to Spring Boot through RTK Query.
- Implement bearer-token attachment and a controlled refresh flow.
- Use Redux DevTools and browser Network tools to inspect state and API calls.
- Diagnose CORS, authentication, serialization and cache problems systematically.

Prerequisites

- Listings and reviews work independently in Postman.
- Frontend pages from Guide 08 compile.

Suggested student time: 2.5-3 hours

1. End-to-end request flow

1. A React component calls an RTK Query hook.
2. fetchBaseQuery reads VITE_API_BASE_URL and the access token from Redux state.
3. The browser sends an HTTP request to Spring Boot with Authorization: Bearer <token>.
4. Spring Security validates the JWT, then Controller -> Service -> Repository -> MySQL executes.
5. The backend returns a DTO as JSON; RTK Query caches it and React re-renders.
6. Mutations invalidate tags so affected queries refetch automatically.

2. Configure exact CORS origins in Spring Boot

```[Java]
@Bean
CorsConfigurationSource corsConfigurationSource(
    @Value("${app.cors.allowed-origins}") List<String> origins) {
  CorsConfiguration config = new CorsConfiguration();
  config.setAllowedOrigins(origins);
  config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
  config.setAllowedHeaders(List.of("Authorization", "Content-Type", "Accept"));
  config.setExposedHeaders(List.of("Location"));
  config.setAllowCredentials(true);
  config.setMaxAge(3600L);

  UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
  source.registerCorsConfiguration("/api/**", config);
  return source;
}
```

CORS rule: http://localhost:5173 and http://127.0.0.1:5173 are different origins. The configured origin must match the browser address exactly. Do not combine allowCredentials(true) with wildcard origins.

3. Auth slice: keep the access token minimal

```[TypeScript]
interface AuthState {
  accessToken: string | null;
  user: { id: number; email: string; role: string } | null;
}

const initialState: AuthState = { accessToken: null, user: null };

const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    credentialsReceived(state, action: PayloadAction<AuthResponse>) {
      state.accessToken = action.payload.accessToken;
      state.user = action.payload.user;
    },
    signedOut(state) {
      state.accessToken = null;
      state.user = null;
    },
  },
});
```

For the secure baseline, keep the short-lived access token in memory and use an HttpOnly, Secure, SameSite refresh cookie issued by the backend. On a page reload, call /auth/refresh with credentials included. Avoid localStorage for long-lived bearer tokens because injected scripts can read it.

4. RTK Query base query with one refresh attempt

```[TypeScript]
import { Mutex } from 'async-mutex';
import { fetchBaseQuery } from '@reduxjs/toolkit/query/react';
import type { BaseQueryFn, FetchArgs, FetchBaseQueryError } from '@reduxjs/toolkit/query';
import { credentialsReceived, signedOut } from '../features/auth/authSlice';

const mutex = new Mutex();
const rawBaseQuery = fetchBaseQuery({
  baseUrl: import.meta.env.VITE_API_BASE_URL,
  credentials: 'include',
  prepareHeaders: (headers, { getState }) => {
    const token = (getState() as RootState).auth.accessToken;
    if (token) headers.set('authorization', `Bearer ${token}`);
    return headers;
  },
});

export const baseQueryWithReauth: BaseQueryFn<
  string | FetchArgs, unknown, FetchBaseQueryError
> = async (args, api, extraOptions) => {
  await mutex.waitForUnlock();
  let result = await rawBaseQuery(args, api, extraOptions);

  if (result.error?.status === 401) {
    if (!mutex.isLocked()) {
      const release = await mutex.acquire();
      try {
        const refresh = await rawBaseQuery(
          { url: 'auth/refresh', method: 'POST' }, api, extraOptions);
        if (refresh.data) {
          api.dispatch(credentialsReceived(refresh.data as AuthResponse));
          result = await rawBaseQuery(args, api, extraOptions);
        } else {
          api.dispatch(signedOut());
        }
      } finally {
        release();
      }
    } else {
      await mutex.waitForUnlock();
      result = await rawBaseQuery(args, api, extraOptions);
    }
  }
  return result;
};
```

```
[Terminal]
npm install async-mutex
```

5. Complete Redux store configuration

```
[TypeScript]
export const store = configureStore({
  reducer: {
    auth: authReducer,
    [baseApi.reducerPath]: baseApi.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(baseApi.middleware),
  devTools: import.meta.env.DEV,
});
```

Store rule: Add baseApi.middleware exactly once. Adding both the base API and an injected API middleware duplicates the same middleware and causes warnings or repeated behavior.

6. Install and use Redux DevTools in the browser

7. Install the official Redux DevTools extension from the browser extension store linked by the Redux documentation.
8. Restart the browser if required and open the React application.
9. Open Developer Tools and select the Redux tab.
10. Inspect auth/credentialsReceived after login, API query cache entries, pending/fulfilled/rejected actions and tag invalidation.
11. Use action/state inspection for development only. Never expose real production tokens during demonstrations or screen recordings.
12. Redux Toolkit configureStore enables the extension automatically in development; no legacy composeWithDevTools code is needed.

7. Test APIs from the frontend

13. Start MySQL, then Spring Boot on 8080, then Vite on 5173.
14. Open Browser DevTools > Network and preserve the log.
15. Log in and verify a 200 response. Confirm the access token is not written to console or localStorage.
16. Create a listing. Verify POST /api/v1/listings, Authorization header, request JSON and 201 response.
17. Observe Redux actions and automatic refetch of the listings query.
18. Update/delete as owner and attempt the same operation from another user to confirm the backend returns 403.
19. Create a review for a seeded completed order and verify duplicate review returns 409.

8. Error-to-fix matrix

Observed symptom	Evidence to inspect	Most likely fix
Browser says CORS blocked	Console and OPTIONS request	Match APP_ALLOWED_ORIGINS; enable cors() in SecurityFilterChain; allow method/header.
401 although login worked	Authorization header and token expiry	Verify prepareHeaders and token state; check issuer/secret/clock.
Infinite refresh requests	Network waterfall and Redux actions	Refresh endpoint must not use the same failing loop; allow one retry; use mutex.
403 on owner action	JWT subject and database owner email/id	Use stable user ID/subject consistently; verify ownership rule.
400 with no useful form message	Backend ApiError and UI mapping	Map fieldErrors to form fields and display generic fallback.
Response field undefined	Network JSON versus TS interface	Align DTO field names; regenerate/update types.

Observed symptom	Evidence to inspect	Most likely fix
Stale list after create/update	Redux cache/tags	Provide and invalidate matching Listing tags.
Duplicate requests in development	React StrictMode or duplicate middleware	Understand StrictMode behavior; ensure middleware registered once.
Failed to fetch	Backend down, wrong VITE URL or mixed content	Check server/port; restart Vite after env change; use HTTPS consistently.
MySQL lazy serialization error	Entity leaked outside transaction	Return mapped DTOs; fetch required relationships explicitly.

9. Integration acceptance tests

- Unauthenticated users can browse listings but cannot create/edit/archive or review.
- A seller can create and manage only their listings.
- A buyer can review only their own completed order and only once.
- A mutation updates the relevant UI without a manual full-page refresh.
- Expired access token triggers at most one refresh attempt, then either retries successfully or signs out.
- All errors produce understandable user feedback while technical details remain in development tools/logs.

Completion checklist

☐ CORS is exact and credential-aware.

☐ baseApi attaches bearer tokens and uses credentials only for refresh cookie.

☐ Redux store includes API reducer and middleware once.

☐ Redux DevTools shows actions and RTK Query cache.

☐ Network panel confirms correct methods, URLs, status codes and JSON.

☐ The full Listings and Reviews flows pass with two different user accounts.